

Phase 5, Analysis of Provided Results

Annexes F-I, plus my own execution evidence Samuel, QA Automation Senior · 15 August 2026
· v1.0

⚠ **Revised 15/08/2026 after live verification.** A browser pass established that the demonstration backend is a json-server fixture, which re-attributes six of the findings below from *product defect* to *environment artefact*, and hardens four new ones into Critical product defects. The classification table in §1 is superseded by docs/08-live-verification.md §7. The analysis of Annexes F-I is unaffected and stands as written.

The brief warns that some of the supplied data reveals anomalies and some does not. Distinguishing the two is the exercise. Below, each observation is classified as **product defect**, **test defect**, **data problem**, or **environment problem**, and where the classification cannot be made with the evidence available, I say so and state what would settle it.

Part 1, Classification summary

#	Observation	Source	Classification	Severity
1	Anonymous read of /users, /contacts, /transactions	My run	☐ Product defect	Blocker
2	?userId= accepted and ignored	My run + H TC-12	☐ Product defect	Blocker
3	Member gets 200 from /admin/users	H TC-11	☐ Product defect	Blocker
4	\$1M transfer accepted, 201	H TC-03	☐ Product defect	Critical
5	Two keyless POSTs both settled	G-B	☐ Product defect	Critical
6	Balance KPI ≠ table total (\$3,491)	F.6	☐ Product defect (<i>pending Q2</i>)	Major
7	amount float-dollars vs contract cents-integer	My run + E	☐ Product/contract defect	Major
8	Out-of-enum filter → 200 []	My run	☐ Product defect	Major
9	No /health, /ready, /metrics	My run + D	☐ Product defect	Major
10	No correlation id returned to caller	My run + G	☐ Product defect	Minor
11	TC-02 scheduled date off by one day	H	◦ Test defect	—

#	Observation	Source	Classification	Severity
12	TC-04 timeout, no assertion	H	◦ Test defect — <i>root cause now confirmed: data-testid=row does not exist; the app renders tx-row</i>	—
13	Demo data ≠ documented data	F.6 vs my run	☐ Data problem	Minor
14	Manager owns zero contacts	My run	☐ Data problem	Minor
15	89 webhook signature failures	I	• Undetermined	—
16	Webhook replayed twice, no-op'd	G-C	☐ Correct behaviour	—
17	Extract A nominal sequence	G-A	☐ Correct behaviour	—
18	transfers_total 2.8% failure rate	I	• Undetermined	—
19	p95 ≈ 200 ms on /transfers	I	☐ Within target	—

Two of the nineteen are test defects, not product defects. That distinction is the most valuable thing in this analysis: acting on #11 and #12 as though they were product bugs would send developers to debug working code.

Part 2, The correlations that matter

2.1 The double-debit chain (Annexe G-B × Annexe I × Annexe E)

Three sources, one story:

Annexe E	→ contract documents NO Idempotency-Key	— the cause
Annexe F	→ header appears in sample requests only	— the illusion
Annexe G-B	→ two POSTs, key=(none), both persisted, both settled 91 ms apart by bank-adapter	— the effect
Annexe I	→ transfers_total{failed}=1377 (2.8%)	— the noise

u3 submits at 10:22:14.891. A second request arrives at 10:22:15.012 — **121 ms later**, under two distinct correlation ids. Both pass validation, both persist, both settle (aa11bb22, cc33dd44).

121 ms is the signature of a **double-click on the submit button**, not a retry and not a deliberate second transfer. A human cannot decide to send twice that fast.

So the chain is: the UI does not disable the button on submit → the client does not attach an Idempotency-Key → the server does not require one → the bank adapter settles both. **Four independent defences absent, in a row.** Each alone would have prevented the loss.

Why the contract matters here: the header exists in Annexe F's examples but in no contract, no rule and no NFR. A guarantee documented only by example is a guarantee no one is accountable for

implementing.

What is missing to close this: whether `transfers_total{failed}=1377` includes rejected duplicates. If duplicates are counted as *successes*, the real double-debit rate is invisible in the metrics, which would mean this has been happening in preprod unnoticed.

2.2 The authorisation chain (my run × Annexe H TC-11 × TC-12)

Annexe H reports two authorisation failures against preprod:

- **TC-11:** MEMBER GET /admin/users → expected 403, got **200 with 8 user records**
- **TC-12:** MEMBER GET /transactions?userId=<other> → expected 403, got **200 with 4 transactions of u2**

I reproduced the same failure class on the demo host, and found it is worse than Annexe H shows:

Test	Annexe H (preprod, authenticated MEMBER)	My run (demo, no credentials at all)
User directory	200, 8 records	200, 3 records
Foreign ledger	200, 4 records of u2	200, all 15 records
Contact book	<i>not tested</i>	200, all 3 owners

The correlation matters here. Two independent environments, two different test suites, same failure mode. This is not a preprod misconfiguration, it is **the absence of an authorisation layer**, reproducible wherever the service runs.

One nuance worth stating: preprod returned **8** user records, the demo returns **3**. Different datasets, same hole. It also means preprod holds more accounts than the three documented, which is worth a question of its own.

2.3 The webhook picture (Annexe G-C × Annexe I), mostly *not* a defect

This is the annexe most likely to be misread as a defect, and I do not think it is one.

```
11:03:22.108 signature malformed → verification=failed → 401   □ correct
11:03:25.442 retry=1, verification=ok                           □ correct
11:03:25.478 transferId=9f2a4c1c → already_settled → noop      □ correct
11:04:11.203 retry=2, verification=ok                           △ why?
11:04:11.234 already_settled → noop                             □ correct
```

What is working: signature verification rejects a malformed signature; the retry with a valid signature succeeds; and, importantly, the settlement is **idempotent**. `already_settled → noop` is exactly right, twice.

The irony is worth naming: **the webhook path handles replay correctly while POST /transfers does not**. The team demonstrably knows how to build this. The capability exists in one service and is missing in the other.

The one genuine question: why does `retry=2` occur at all, 46 seconds after `retry=1` already succeeded with `verification=ok`? Two readings:

- *Benign:* the sender did not receive our 200 (timeout on our side), so it retried per the Annexe D policy (`retry ×5`, exponential back-off). The gaps (3.3 s → 46 s) are consistent with exponential back-off.
- *Defect:* we return 200 too late, or not at all, so the sender never converges.

I cannot tell from a log that does not record our outbound response. That is the gap. Annexe I's `webhook_verification_failures_total = 89` over 24 h does not disambiguate either, 89 failures could be one misconfigured sender retrying, or 89 distinct forgery attempts from different sources. The extract shows `source_ip=203.0.113.42`, a single address, which leans towards a misconfiguration rather than an attack, but one log line is not a distribution.

Proposed action: add the outbound status code and latency to `webhook-svc` logging, and break `webhook_verification_failures_total` down by `source_ip`. Both are one-line changes and both would settle it.

Part 3, Test defects (do not send these to development)

3.1 TC-02, a genuinely broken test

FAIL TC-02 Scheduled transfer displays the requested date
AssertionError: expected 2026-08-15 to equal 2026-08-14

```
test('TC-02 scheduled date', async () => {
  const res = await api.post('/v1/transfers', scheduledPayload);
  const uiDate = await page.getByTestId('transfer-date').innerText();
  expect(uiDate).toEqual('2026-08-14'); // ← hard-coded
});
```

Classification: test defect, with a product question hiding behind it.

Three things are wrong with the test itself:

1. **The expected value is hard-coded** to 2026-08-14. Annexe F.3 shows the payload requests 2026-08-15. The test asserts a date the request never asked for, so it fails on a *correct* system.
2. **res is never asserted.** The POST could return 500 and the test would still proceed to compare UI text. It cannot distinguish “wrong date” from “transfer never created”.
3. **It mixes API and UI in one step with no synchronisation**, no wait for the UI to reflect the new transfer. Inherently racy.

But the product question is real. The expected/actual pair (2026-08-15 vs 2026-08-14) is a **one-day** delta, and Annexe A specifies UTC for technical timestamps and Europe/Paris for display. Paris is UTC+2 in August. A `scheduledDate` stored as 2026-08-15T00:00:00Z and rendered in Paris local time is still the 15th, but stored as 2026-08-14T22:00:00Z (midnight Paris) and rendered in UTC, it displays as the 14th.

So: **fix the test first** (assert against the requested date, assert the POST status, wait for the UI), then see whether it still fails. If it does, there is a real timezone defect on scheduled payments, and a payment executing a day early is a genuine incident.

Recommended rewrite:

```
test('TC-02 scheduled transfer displays the requested date', async () => {
  const requested = '2026-08-15';
  const res = await api.post('/v1/transfers', { ...scheduledPayload, scheduledDate: requested });
  expect(res.status()).toBe(201); // assert the precondition
  await page.goto(`/transactions/${(await res.json()).id}`);
  await expect(page.getByTestId('transfer-date')).toHaveText(requested); // auto-waits
});
```

3.2 TC-04, a test that cannot conclude

ERROR TC-04 GET /v1/transactions?status=pending remains stable under refresh
TimeoutError at transfers.spec.ts:142

Note: no assertion message, runner stopped on timeout

```
test('TC-04 pending list', async () => {
  await page.goto('/transactions?status=pending');
  await page.waitForSelector('[data-testid=row]', { timeout: 2000 });
});
```

Classification: test defect, and possibly masking an environment problem.

- **The test has no assertion.** It navigates and waits for a selector. It cannot fail *meaningfully*, only time out. Its own name promises stability “under refresh” and it never refreshes anything.
- **A 2000 ms timeout is arbitrary and too tight** for a page that must authenticate, route, fetch and render. Annexe I puts /transactions p95 near the 300 ms budget for the API call *alone*; a full page render on a cold SPA can exceed 2 s legitimately.
- **ERROR rather than FAIL** means the runner aborted. Any test scheduled after it in the same file never ran, which is why the report shows Total: 11 but lists TC-01 ... TC-12 with **no TC-10 at all**. One brittle timeout silently reduced the suite.

What is missing to conclude: whether the 2 s was exceeded by 50 ms or by 20 s. The first is a test-tuning problem; the second is an environment problem worth investigating. The report does not say, because the test captured nothing.

Recommended rewrite: assert on content, not on time.

```
test('TC-04 pending list is stable across reloads', async () => {
  await page.goto('/transactions?status=pending');
  const first = await page.getByTestId('row').allInnerTexts();
  expect(first.length).toBeGreaterThan(0);
  await page.reload();
  await expect(page.getByTestId('row')).toHaveText(first); // auto-retries
});
```

Part 4, The numbers, read carefully

4.1 Annexe I latency, the target is met, but the metric is thin

le="0.1"	12034	le="0.5"	47102	
le="0.2"	38221	le="1.0"	47498	
		le="2.5"	47501	count 47503

Cumulative buckets → p50 ≈ **0.2 s**, p95 ≈ **0.5 s**, p99 ≈ **1.0 s**.

Against Annexe D (p50 < 200 ms, p95 < 500 ms, p99 < 1000 ms): **all three sit exactly on the boundary**. p50 lands at the 38221 mark within the 0.2 bucket (23751 is the median of 47503), p95 at 45128 falls in the 0.5 bucket, p99 at 47028 in the 1.0 bucket.

Two caveats before anyone calls this a pass:

1. **Bucket resolution is too coarse to distinguish “meets” from “just misses”.** With no bucket between 0.2 and 0.5, a true p95 of 0.49 s and one of 0.21 s are indistinguishable. I would ask for buckets at 0.05/0.075/0.1/0.25/0.3.
2. **This is a 24-hour aggregate on preprod, not a measurement under the contracted load.** Annexe D specifies “50 req/s for 10 min, iso-prod”. 47,503 requests over 24 h averages **0.55 req/s**, roughly 1% of the target throughput. This tells us nothing about behaviour at 50 req/s.

Verdict: within target at ~1% of the contracted load. Not evidence of meeting the NFR.

4.2 transfers_total, a 2.8% failure rate with no breakdown

completed 41022 pending 5104 failed 1377 → 2.8% failed

Is 2.8% good? **Unanswerable as given**, and that is the finding. There is no declared error budget for transfer *outcomes* (Annexe D's "< 0.1%" is an HTTP error rate, a different measurement), and failed has no reason dimension.

A failed transfer can be: insufficient funds (correct business behaviour), a rejected duplicate (correct, and desirable), a bank timeout (a resilience problem), or a validation rejection (correct). **These demand opposite responses**, and the counter collapses them into one number.

Proposed action: add a reason label to transfers_total{status="failed"}. Until then, 2.8% is not interpretable, and, per §2.1, if rejected duplicates are counted as completed, the double-debit rate is structurally invisible.

4.3 Annexe F.6, the documented data does not exist

	Documented (F.6)	Live (my run)
Dates	09-12/08/ 2026	01-22/07/ 2025
Labels	"Loyer juillet", "Retrait DAB"	"Wire Transfer to Sarah", "ATM Withdrawal"
Table total	\$44,800	-\$389.66 (all) / \$1,883.14 (completed)
Balance KPI	\$48,291	(unverified — no UI access)

Classification: data problem, not a product defect. The documentation is stale relative to the deployed environment, normal in a real project, and exactly why my automated assertions are invariant-based rather than value-pinned.

But the F.6 gap is still a real signal. A \$44,800 table total against a \$48,291 KPI is a **\$3,491** difference with no stated derivation. Whatever data was loaded when that was captured, the two figures disagreed. That is DEF-010, and it is why PO question Q2 exists.

Part 5, Defect reports

DEF-001, Unauthenticated read access to the user directory and contact book

Field	Value
ID	DEF-001
Title	GET /api/v1/users and /contacts return complete records to a caller with no credentials
Severity	Blocker
Priority	P1 — immediate
Component	api-gateway / authorisation
Environment	Demo — http://149.56.128.57:3200, 15/08/2026
Found by	Exploratory session, confirmed by TS-01 / TS-01b
Rules breached	R10; Annexe D ("données sensibles")

Steps to reproduce

1. Open a client with **no** session cookie and **no** Authorization header.
2. GET http://149.56.128.57:3200/api/v1/users
3. GET http://149.56.128.57:3200/api/v1/contacts

Expected: 401 Unauthorized. R10 restricts the user directory to ADMIN; a caller presenting no identity cannot satisfy any role check.

Actual: 200 OK with the full payload, both endpoints.

// GET /api/v1/users → 200

```
[ { "id": "u1", "name": "Amara Kofi", "email": "admin@nexapay.test", "role": "ADMIN" },
  { "id": "u2", "name": "Fatou Diallo", "email": "manager@nexapay.test", "role": "MANAGER" },
  { "id": "u3", "name": "Kwame Mensah", "email": "member@nexapay.test", "role": "MEMBER" } ]
```

// GET /api/v1/contacts → 200 (every owner, not just the caller's)

```
[ { "id": "c1", "name": "Sarah Chen", "accountMask": "****4829", "ownerId": "u3" },
  { "id": "c2", "name": "James Osei", "accountMask": "****7731", "ownerId": "u1" },
  { "id": "c3", "name": "Lina Vasquez", "accountMask": "****2154", "ownerId": "u1" } ]
```

Impact

- **Personal-data disclosure.** Names and e-mail addresses of every user: plus the counterparties they transact with, to anyone who can reach the host. Under GDPR this is a reportable breach, not an internal issue.
- **Privilege reconnaissance.** The role field names the ADMIN account: so an attacker knows exactly which credential to target before attempting anything.
- **Social-engineering material.** Knowing that Kwame pays Sarah Chen at ****4829 is precisely what a convincing payment-redirection fraud needs.
- **Combined with DEF-003:** an unauthenticated party can enumerate users, their contacts, and the full transaction ledger, the complete relationship graph of the business.

Hypotheses (ranked)

1. **No authentication middleware on the API gateway at all**, most likely. Consistent with DEF-002 and DEF-003 failing identically, and with Annexe H TC-11/TC-12 failing on a *different* environment.
2. Auth middleware present but not applied to these routes, a route-registration omission.
3. A deliberate demo-environment concession, **this is PO question Q4.**

What is missing to conclude: whether production shares this configuration. Hypothesis 3 would downgrade this to an environment problem; hypotheses 1 and 2 keep it a Blocker. **This is the single question with the most leverage in the whole assessment.**

Evidence: evidence/html-report/, TS-01, TS-01b attachments (full request/response). Fixtures with provenance in src/fixtures/recorded/.

Proposed action: confirm production configuration **today**. If reproducible outside the demo, treat as a security incident: rotate nothing (no credentials leaked) but close the endpoint before any further exposure.

DEF-003, No tenant scoping on the transaction ledger

Field	Value
ID	DEF-003
Title	GET /transactions returns the global ledger; the userId parameter is accepted and ignored
Severity	Blocker
Priority	P1
Component	transactions service / authorisation

Field	Value
Found by	TS-02 — corroborates Annexe H TC-12

Steps to reproduce

1. With no credentials: GET /api/v1/transactions → 200, **15 records**.
2. GET /api/v1/transactions?userId=u2 → 200, **the same 15 records, byte-identical**.
3. GET /api/v1/transactions?userId=does-not-exist → 200, same 15 records.

Expected: records scoped to the authenticated caller; a foreign userId refused with 403 or scoped to the caller regardless.

Actual: the full ledger, unscoped, for any caller. The parameter has no effect whatsoever.

Impact Full financial history, counterparties, amounts, dates, statuses, exposed to any caller. The userId parameter creates a **false impression of scoping** in the API surface, which is arguably worse than having no parameter: a reviewer reading client code that passes userId will reasonably assume filtering occurs.

Correlation: Annexe H TC-12 reports the same failure on preprod with an *authenticated* MEMBER receiving 4 transactions belonging to u2. Two environments, two suites, one root cause.

Hypotheses

1. Scoping was never implemented server-side; the UI filters client-side. Highly likely, it would explain why the parameter exists but does nothing.
2. Scoping depends on a JWT claim that is absent when unauthenticated, and the code fails open rather than closed.

What is missing: the behaviour with a *valid* MEMBER token on this environment. Annexe H suggests it is equally broken; I could not verify it here.

Proposed action: derive scope from the authenticated principal server-side and **remove** the userId parameter entirely. A scoping parameter the client can set is not a scoping mechanism.

DEF-005, Duplicate transfer settlement (no idempotency enforcement)

Field	Value
ID	DEF-005
Title	Two POSTs 121 ms apart with no Idempotency-Key both persist and both settle
Severity	Critical
Priority	P1
Component	transfer-svc / bank-adapter / web client
Found by	Annexe G extract B (log analysis)

Steps to reproduce (*inferred, not yet executed against the demo host*)

1. Sign in as a MEMBER and open the transfer form.
2. Complete a valid transfer.
3. **Double-click** the submit button.

Expected: one debit. Either the client disables the control on submit, or the server deduplicates on an idempotency key.

Actual (Annexe G-B, verbatim):


```

10:22:14.891 [api-gateway] corr-f4a2b1c8 POST /v1/transfers user=u3 received
10:22:14.912 [transfer-svc] corr-f4a2b1c8 idempotencyKey=(none) validation=ok
10:22:14.934 [transfer-svc] corr-f4a2b1c8 transferId=aa11bb22 status=pending persisted
10:22:15.012 [api-gateway] corr-8b3d4e2a POST /v1/transfers user=u3 received ← +121 ms
10:22:15.031 [transfer-svc] corr-8b3d4e2a idempotencyKey=(none) validation=ok
10:22:15.049 [transfer-svc] corr-8b3d4e2a transferId=cc33dd44 status=pending persisted
10:22:17.611 [bank-adapter] transferId=aa11bb22 settled
10:22:17.702 [bank-adapter] transferId=cc33dd44 settled

```

Impact The customer is debited twice. Recovery requires manual reconciliation and a refund, operational cost, customer trust, and on a regulated product a reportable incident. At scale this is the defect that generates chargebacks.

Analysis 121 ms is too fast for a human decision and too fast for a typical retry back-off. It is the signature of a double-click. Four defences were absent simultaneously:

Layer	Defence	Present?
Client	Disable submit on click	☐
Client	Send Idempotency-Key	☐ ((none))
Server	Require the key	☐
Server	Deduplicate by payload+window	☐

Correlation: Annexe G extract C proves webhook-svc handles replay correctly (already_settled → noop, twice). **The organisation knows how to do this.** The capability exists in one service and is missing in the one that moves money.

What is missing: whether transfers_total{failed}=1377 (Annexe I) counts rejected duplicates. If duplicates are counted as completed, the true rate of this defect is invisible in monitoring.

Proposed action (in order of cost)

1. Disable the submit control on click, hours, removes the common case.
2. Make Idempotency-Key **mandatory**, return 400 without it, days.
3. Server-side dedup window on (user, recipient, amount, hash), the real fix.
4. Add the key to the OpenAPI contract so it stops being folklore.

DEF-007, Business ceiling not enforced server-side

Field	Value
ID	DEF-007 · Severity Critical · Priority P1
Component	transfer-svc validation
Found by	Annexe H TC-03; TS-04 written, unverified from my runner

Steps: POST /v1/transfers with amount: 1000000000 (cents = \$1,000,000), all other fields valid.

Expected: 400, R03 caps a transfer at \$9,999.99. **Actual:** 201, id=3e91c0aa-17b4-4d2e-9c11-0b77d2a1f008, status=pending.

Impact: a transfer 100× the documented ceiling was accepted for settlement. Any non-browser caller, curl, a script, a compromised client, bypasses the limit entirely.

Root cause is documented, not hidden. R03 states the rule applies “validation côté formulaire”. Annexe E sets no maximum on amount. **The ceiling was specified as a client-side control**, which on a payments API means it is not a control at all.

Proposed action: add maximum to the schema and enforce it in transfer-svc; re-run TS-04's 10 boundary cases. Then audit R01, R02 and R04 the same way — they carry the same “côté formulaire” wording and are likely to have the same gap.

DEF-010, Balance KPI not derivable from the ledger

Field	Value
ID	DEF-010 · Severity Major (<i>provisional — depends on Q2</i>) · Priority P2
Component	dashboard KPI computation
Found by	Annexe F.6; TS-08b written, blocked

Observed (Annexe F.6): table foot **\$44,800**, Balance KPI **\$48,291** — a **\$3,491** gap, no stated derivation.

On the live environment the documented figures do not reproduce at all. From the 15 served records I compute:

Derivation	Value
All statuses	-\$389.66
Completed only	\$1,883.14
Income (positive)	\$9,052.49
Expenses (negative)	\$9,442.15

Impact: operators authorise transfers based on the balance they can see. If the KPI includes pending or failed movements, it **overstates available funds** and directly causes transfers against money that has not settled. That is a financial-integrity defect wearing a display-defect costume.

Why the severity is provisional: if the intended derivation is completed-only, this is a P2 display bug. If pending movements are included, it is a P1 integrity defect. **The severity swings two levels on the answer to Q2**, which is why the automated test refuses to guess and reports both candidate derivations in its failure message instead.

What is missing: the specification. Nowhere in the brief is “Total Balance” defined.

Part 6, What I could not determine, and what would settle it

Stated explicitly, because a report that hides its own gaps is not useful.

Open question	Why it is unresolved	What would settle it	Effort
Does DEF-001 exist in production?	No production access	PO/DevOps confirmation of gateway config (Q4)	5 min
Is POST /transfers validation broken on <i>this</i> host?	No successful create from my runner	Run the 14 written tests with TEST_MODE=live	10 min
Is the balance KPI wrong, or am I?	Derivation unspecified	Q2	5 min
Are the 89 webhook failures an attack or a misconfiguration?	No source_ip breakdown	Label the metric by source	1 h

Open question	Why it is unresolved	What would settle it	Effort
Is TC-02 a timezone defect once fixed?	Test is broken first	Fix the test, re-run	30 min
Does trans-fers_total{failed} include duplicates?	No reason label	Add a reason dimension	2 h
Why does the webhook sender retry after success?	Outbound response not logged	Log status + latency in webhook - svc	1 h

Every item above is answerable in under two hours of someone else's time. In practice that means the blocking constraint is not engineering effort, it is five unanswered questions.

Part 7, My own execution evidence

Suite: Playwright + TypeScript · TEST_MODE=recorded · run final-001

42 tests 9 passed 9 failed 24 skipped 41.8 s
of which the blocking API gate: 32 tests 9 passed 9 failed 14 skipped

Verdict	Count	Meaning
☑ Passed	9	Invariants and schemas hold on the served data
☑ Failed	9	Reproduced defects — every one verified live
◦ Skipped	24	14 × POST /transfers (never observed) + 10 × browser-backed (no UI in replay mode)

The 9 failures, each a real finding:

Test	Defect
TS-01, TS-01b	DEF-001 — anonymous directory + contact read
TS-02	DEF-003 — no tenant scoping
TS-06b	DEF-004 — money typing contradicts the contract
TS-09	DEF-009 — out-of-enum filter returns 200 []
TS-10c ×3	DEF-011 — /health, /ready, /metrics all 404
TS-10d	DEF-012 — no correlation id returned

On the skips, this is deliberate and it is the point. The replay server returns 501 not_recorded for POST /transfers, because that endpoint was never successfully exercised. A permissive mock would have made all ten “must be rejected” tests **pass** and manufactured false confidence. A strict mock would have made them **fail** and manufactured ten defects nobody could reproduce. Both are fabrication.

24 skips is the honest measure of what this environment let me verify, and it is why the Go/No-Go recommendation is bounded rather than confident. Ten of them are the browser-backed tests, skipped because replay mode serves API payloads and has no UI to drive, a mocked UI test that always passes would be worse than no test at all.

Artefacts: evidence/html-report/index.html (per-call request/response, redacted), evidence/junit.xml, evidence/results.json, evidence/traces/.

Recommendation in docs/06-go-no-go.md.